

GPU-Sentry: Instruction-Level Semantic Detection of GPU Cryptomining Workloads

Pin-Yi Kuo

National Tsing Hua University
Hsinchu, Taiwan
yikuo@lsalab.cs.nthu.edu.tw

Jerry Chou

National Tsing Hua University
Hsinchu, Taiwan
jchou@lsalab.cs.nthu.edu.tw

ABSTRACT

Unauthorized GPU cryptomining consumes expensive accelerator resources and can interfere with legitimate HPC and AI workloads. Existing GPU cryptojacking defenses primarily rely on indirect evidence such as network traffic, host behavior, or GPU telemetry. These signals can reveal abnormal activity, but they do not directly inspect the GPU kernels submitted for execution.

We present GPU-Sentry, a system for instruction-level semantic detection of GPU cryptomining workloads. GPU-Sentry transparently captures GPU code objects and kernel-launch events through CUDA driver interception, reconstructs and normalizes the submitted GPU assembly, and builds stream-level execution windows over the kernel launch sequence. A sequence classifier then detects mining-like computation from normalized instruction traces. The capture layer is deployed system-wide by the administrator and monitors native and containerized CUDA workloads without requiring user-side application changes.

GPU-Sentry is designed around a synthetic-to-real evaluation setting: the detector is trained on synthetic mining and benign kernels and evaluated on real-world GPU captures. We show that normalized GPU instruction traces carry evidence of cryptomining that transfers from synthetic training to real-world captures, and that this evidence stays visible under launch-rate throttling and mixed workloads where a behavior-based GPU-telemetry baseline degrades.

CCS CONCEPTS

• **Security and privacy** → **Systems security**; *Operating systems security*; • **Computer systems organization** → **Heterogeneous (hybrid) systems**.

KEYWORDS

GPU security, cryptomining detection, CUDA, GPU assembly, SASS, runtime monitoring, machine learning

1 INTRODUCTION

GPUs are among the most expensive and contended resources in modern computing infrastructure, which makes them an attractive target for cryptomining abuse. Clusters and cloud GPU pools run workloads that are long-lived and compute intensive, so an unauthorized miner can quietly consume accelerator resources while looking much like an ordinary compute-bound job. The financial incentive is direct: the miner profits while the operator pays for hardware that should be serving legitimate research work.

Existing GPU cryptojacking defenses infer mining from *indirect* evidence rather than from the computation that is actually submitted to the GPU. They watch network traffic, host behavior, GPU telemetry [14, 18], or physical side channels [1, 23]. Such signals can show that a GPU is busy or that kernels launch repeatedly, but they do not answer the security question directly: whether the submitted GPU code implements a mining search loop or a legitimate high-throughput kernel.

We present *GPU-Sentry*, a system for instruction-level detection of cryptomining-like GPU computation. GPU-Sentry transparently captures GPU code objects and kernel-launch events at the CUDA Driver API boundary, reconstructs and normalizes the submitted GPU assembly (SASS, the native instruction set that NVIDIA GPUs execute), and builds stream-level instruction windows over the kernel launch sequence. A ModernBERT [22] sequence classifier then labels each normalized window as mining-like or benign.

The central insight is that mining stays visible in the instruction stream even when host-level evidence is weak. A user can rename processes, obfuscate the host binary, throttle the launch loop, or co-run mining beside benign jobs; these evasions alter names, timing, and surrounding activity, but the GPU must still receive the executable kernels that perform the mining computation. GPU-Sentry therefore treats the submitted GPU program as the primary evidence; device telemetry serves only as a comparison baseline in our evaluation. Table 1 summarizes how this evidence differs from that exposed by prior detection approaches. To the best of our knowledge, GPU-Sentry is the first system to detect GPU cryptomining from the GPU instructions (normalized SASS) actually submitted to the device, rather than from indirect signals such as GPU telemetry, performance counters, or physical side channels.

Building a usable instruction-level detector raises several practical challenges. GPU code arrives in many forms: PTX, fatbins, or CUBINs, from files or memory. Users submit it through diverse libraries, frameworks, and containers, so it must be captured however it is submitted, without user intervention. The same computation compiles to different SASS across GPU architectures, compiler versions, and optimization levels. A miner may launch one kernel repeatedly or split its work across helper kernels, so single launches are not the right unit of analysis. And public GPU-miner datasets are scarce. We develop these challenges and the mechanisms that address them in section 3.

GPU-Sentry addresses these challenges with the following contributions:

- a transparent CUDA Driver API capture layer that records GPU code objects and launch events for system-wide native and containerized workloads;

Table 1: Detection approaches expose different evidence. GPU-Sentry inspects the submitted GPU computation and attributes it to CUDA execution context. (✓ yes, × no, ~ partial)

Capability	Network / flow	Behavioral telemetry	Physical side channel	Static binary analysis	GPU-Sentry
Sees the submitted GPU computation	×	×	×	~	✓
Robust to process or kernel renaming	✓	✓	✓	×	✓
Robust to encrypted network traffic	×	✓	✓	✓	✓
Robust to host binary obfuscation	✓	✓	✓	×	✓
Robust to throttled mining	×	×	×	✓	✓
Robust to mixed benign & mining on the same GPU	×	×	×	✓	✓
Pinpoints the mining process	~	×	×	✓	✓
No special hardware or sensors required	✓	✓	×	✓	✓

- a normalized SASS representation with control-flow-driven main-loop extraction and architecture-aware opcode-family mapping;
- a per-stream window scheduler that assembles launches into model-sized windows and deduplicates them by kernel-composition signature;
- a ModernBERT detector that classifies each normalized instruction window as mining-like or benign; and
- a synthetic-to-real training and evaluation workflow that trains the detector entirely on synthetic normalized SASS and shows that this training transfers to real-world GPU captures.

2 BACKGROUND AND THREAT MODEL

2.1 CUDA Code and Execution

CUDA applications reach the GPU through a layered software stack, shown in figure 1. An application may call the CUDA Runtime API directly, use a CUDA library such as cuBLAS or cuDNN, or run on top of a framework such as PyTorch or TensorFlow. These layers ultimately use the CUDA Driver API [11] to load code modules, resolve kernel handles, and launch kernels on CUDA streams. A stream is an ordered execution queue, so launches issued on the same stream preserve program order even when other streams run concurrently. The CUDA Driver API is therefore the narrow waist that every CUDA application must traverse, regardless of the higher-level interface the application uses.

GPU code appears in several representations before execution. PTX is a virtual instruction set that is just-in-time compiled for a target GPU. SASS is the native instruction set of a specific GPU architecture and the final representation that NVIDIA hardware executes. A CUBIN is a binary for a single GPU architecture, and its machine code is SASS; a fatbin bundles one or more CUBINs, often alongside PTX, so the loader can select the variant matching the GPU in use.

2.2 Threat Model

GPU-Sentry assumes a monitored host whose administrator controls the CUDA driver deployment and the GPU-Sentry collector. An unprivileged user may run arbitrary CUDA applications on the host or inside a container. The user’s miner uses CUDA and is initiated through the system’s CUDA software stack.

The user actively hides the miner’s intent but cannot modify the parts of the system we trust. Specifically, the user may rename processes and kernels, strip symbols, pack or obfuscate the host binary,

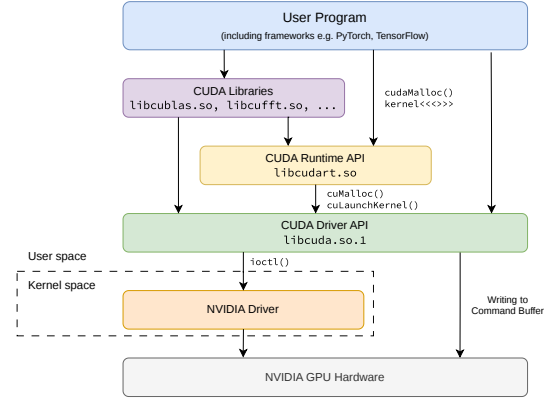


Figure 1: The CUDA software stack. Applications, libraries, the Runtime API, and frameworks all reach the GPU through the CUDA Driver API, which loads code objects and launches kernels.

compile a custom miner, change optimization levels, throttle the kernel-launch loop, and co-run mining with benign GPU work. We assume the user cannot control the host administrator, the kernel-mode driver, or the GPU-Sentry collector. CPU-only mining is out of scope because it never reaches the GPU. We also assume the user does not replace or bypass the monitored userspace CUDA driver that GPU-Sentry interposes on; because that driver is a userspace library, a same-privilege user could in principle do so, a deployment limitation we discuss in section 7.

3 MOTIVATION AND DESIGN CHALLENGES

3.1 Telemetry Is Not Enough

Telemetry-based detection is easy to deploy but cannot identify the submitted computation. Device counters—GPU utilization, memory use, power draw, and temperature—can flag a miner that runs alone on an otherwise idle or lightly loaded GPU, but two effects break this in shared HPC and AI clusters. First, many benign GPU applications are themselves resource-intensive, so high utilization is not distinctive. Second, a miner can move its device-level footprint into ordinary-looking ranges by throttling its launch loop or running beside benign work. In these cases telemetry describes the *effect* of execution, not the *code* that produced it, which makes it prone to both false positives and false negatives.

Our reimplementation of GPU-counter detection following Pott et al. [14]—a Random Forest over whole-device measurements—shows this. It is highly accurate on ordinary isolated runs, but its accuracy drops sharply once the miner throttles its kernel-launch loop or runs beside a benign job (section 6). Throttling spreads the mining activity thinly over time, and a co-running benign job hides it behind its own heavy device usage. GPU-Sentry instead reads normalized SASS windows, so throttling changes only *when* a window fills, not the kernel body that is eventually analyzed.

3.2 Design Challenges

Building an instruction-level detector raises six practical challenges, listed below.

- **Code-object diversity.** GPU code arrives as PTX, fatbins, or CUBINs, from files or memory. GPU-Sentry hooks all Driver-API module and library loads and captures the SASS actually submitted to the device (section 4.2).
- **Transparent capture across workloads.** Workloads span native programs, libraries, frameworks, and containers. GPU-Sentry interposes a replacement `libcuda.so.1` at the Driver-API boundary, propagated into containers through the NVIDIA Container Toolkit (section 4.2).
- **Repeated and split kernels.** A miner may launch one kernel repeatedly or split work across helper kernels. GPU-Sentry analyzes stream-level instruction windows, each filled to the token budget and deduplicated by kernel-composition signature (section 4.4).
- **SASS variation.** SASS is architecture-specific: newer GPU generations add new instructions and rename or retire others. GPU-Sentry normalizes operands and maps architecture-specific opcodes to common semantic families (section 4.3).
- **Compiler and optimization variation.** Code differs across compiler versions and optimization levels. GPU-Sentry combines the normalized representation with multi-optimization (O2/O3) training and leakage-aware splits (section 5).
- **Scarce cryptominer training data.** There is no public GPU-miner dataset and few collectable real miners. GPU-Sentry trains on synthetic workloads and reserves all real captures for evaluation (section 5).

4 SYSTEM DESIGN

4.1 Overview

GPU-Sentry turns opaque CUDA activity into auditable instruction-level evidence. Figure 2 shows the architecture. A client-side CUDA interception library captures code-load and launch events and forwards them asynchronously to the collector, keeping the application-facing CUDA path lightweight. The server-side analysis pipeline then reconstructs code objects, normalizes their SASS, builds stream-local launch windows, renders each window as an instruction sequence, and sends the sequence to the ModernBERT detector. The detector returns a mining score and a verdict that can be logged, reported, or used to terminate the offending process under an administrator-defined policy. Captured artifacts are archived to disk for later audit.

The design separates capture from analysis because CUDA calls are latency-sensitive. The interception path only creates compact

events and places them in an in-process ring buffer; a background thread batches them to the collector. Network transport to the collector, disassembly, normalization, window scheduling, and inference all run off the application’s critical path.

4.2 Transparent CUDA Activity Capture

Every CUDA miner must eventually load GPU code and launch kernels through the CUDA software stack, so GPU-Sentry captures activity at the CUDA Driver API. As section 2 describes, the Driver API is the narrow waist that every CUDA application must traverse, whatever higher-level interface it uses, whether the Runtime API, a CUDA library, or a framework. We interpose with a wrapper library rather than dynamic GPU binary instrumentation such as NVBit [20] or SASSI [16]: per-instruction instrumentation is far heavier than continuous monitoring needs and is awkward to deploy across arbitrary containerized workloads, whereas recording code-load and launch events at the driver-library boundary gives transparency, container deployability, and low overhead.

Wrapper library. Most CUDA Driver API interception tools, including the official CUDA samples [6], install their hook with `LD_PRELOAD` (figure 3a). That mechanism is easy to evade, since a user can simply unset the environment variable, and it is not transparent to containers, because the hook library must also be injected inside each container. GPU-Sentry instead uses a wrapper library (figure 3b) that replaces the system `libcuda.so.1` on the host with its hook, which cannot be disabled by clearing an environment variable. The replacement exposes the expected driver symbols. The original driver library is embedded in the wrapper and extracted and loaded in memory at startup so that intercepted calls can resolve and forward to the real implementation. The wrapper is statically linked and self-contained, so it also runs inside containers: because the NVIDIA Container Toolkit bind-mounts the host’s `libcuda.so.1` into each GPU container, replacing the library once on the host transparently propagates the wrapper to native and containerized workloads alike.

Trampoline symbols. Symbols that GPU-Sentry does not need to observe are served by trampolines that jump directly to the original implementation with a single `jmp` instruction. These calls keep the original calling convention and incur only that one jump, so the common CUDA path stays fast.

Capturing GPU code objects. A CUDA program makes its GPU code available to the device through the module-load and library-load Driver API calls before it can launch any kernel from that code. GPU-Sentry therefore hooks every one of these load paths, listed in the upper block of table 2. The captured input may be file-backed or in-memory, and may be a CUBIN, a fatbin, or null-terminated PTX; capture is agnostic to the form, identifying the code type from its magic bytes and reading the object’s header to recover its size, which the in-memory load APIs do not provide.

Associating kernel-launch handles. The kernel-launch Driver API identifies a kernel only by an opaque function handle, not by its name or its code. GPU-Sentry must therefore resolve each handle back to its kernel name and to the module or library that supplied its code. It hooks the handle-resolution and launch entry points in the lower blocks of table 2, mapping each function handle

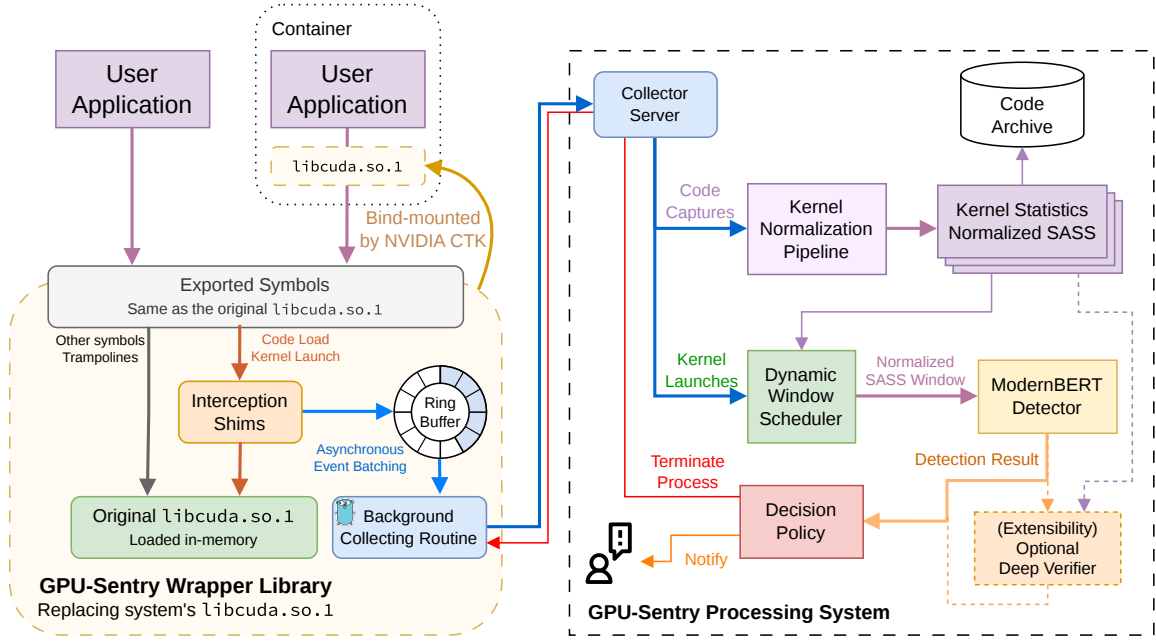


Figure 2: GPU-Sentry architecture. The interception library records CUDA’s module code load and launch activity; the server-side analysis pipeline reconstructs normalized SASS, builds stream-level windows with the window scheduler, and invokes the ModernBERT detector, which produces a verdict. An optional verification layer (dashed) can be added but is not used in our evaluation.

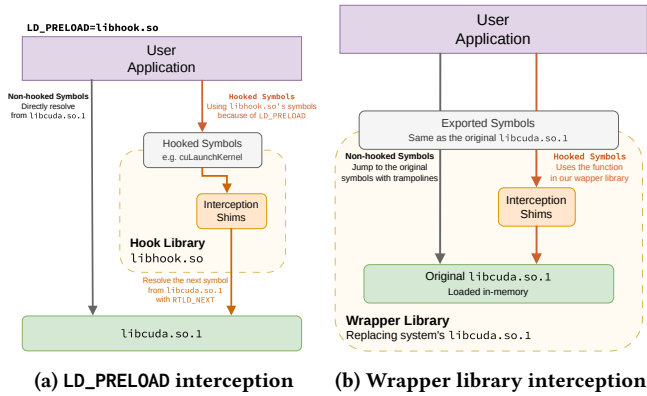


Figure 3: LD_PRELOAD interception versus GPU-Sentry’s wrapper-library deployment. LD_PRELOAD requires an additional hook library to be injected ahead of the CUDA driver, while the wrapper replaces the system libcuda.so.1 and embeds the original driver.

to its module or library and thus to the captured code object (figure 4). Each launch event carries process identity, device, stream, function handle, code identifier, launch dimensions, and timestamp.

Asynchronous buffering. The interception path does the minimum work on the application thread: it records a small event holding only the call’s metadata, such as handles, identifiers, launch dimensions, and a timestamp, and pushes it into an in-process ring

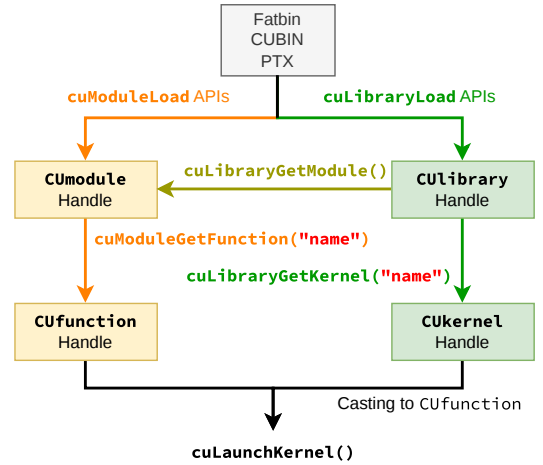


Figure 4: Associating a launched kernel to its SASS. Handle-resolution hooks map a function handle to a module or library, and launch hooks resolve each launch to the captured code object.

buffer. A background thread batches buffered events and ships them to the collector as Protobuf messages. If the ring buffer fills faster than the background thread can drain it, new events are dropped rather than blocking the application thread, so capture latency stays low even under high launch rates.

Table 2: CUDA Driver API entry points intercepted by GPU-Sentry, grouped by role.

Driver API	Purpose
<i>Code-object load</i>	
cuModuleLoad	Load a module from a file.
cuModuleLoadData	Load a module from an in-memory image.
cuModuleLoadDataEx	Load a module from memory with JIT options.
cuModuleLoadFatBinary	Load a module from a fat binary.
cuLibraryLoadData	Load a library from an in-memory image.
cuLibraryLoadFromFile	Load a library from a file.
cuLibraryGetModule	Retrieve the code module from a loaded library.
<i>Kernel-handle resolution</i>	
cuModuleGetFunction	Resolve a kernel handle from a module.
cuLibraryGetKernel	Resolve a kernel handle from a library.
<i>Kernel launch</i>	
cuLaunchKernel	Launch a kernel on a stream.
cuLaunchKernelEx	Launch a kernel with extended attributes.
cuLaunchCooperativeKernel	Launch a cooperative kernel.

4.3 Kernel Code Normalization Pipeline

GPU-Sentry analyzes SASS, the final representation the device executes: every CUDA program, mining or benign, must ultimately produce SASS to run, whereas higher-level forms such as PTX are optional and can be stripped or absent. Analyzing SASS therefore guarantees coverage of any submitted computation after host compilation, library loading, and PTX JIT translation. Raw SASS, however, is not directly usable for detection: it is specific to a compiler, architecture, and binary instance, and a single kernel often contains regions—initialization, error handling, teardown—that are irrelevant to what it computes. GPU-Sentry therefore passes every captured code object through a normalization pipeline before detection. Code captured as PTX is first compiled to target SASS. Each object is then disassembled [10], split into per-kernel instruction sequences, and passed through control-flow analysis: short kernels keep their full normalized sequence, while long kernels are reduced to the main computation loop or compute-heavy section recovered from the control-flow graph, so the classifier focuses on the recurring computational work.

Algorithm 1 summarizes the loop selection. GPU-Sentry treats every backward branch—a branch that jumps back to an earlier instruction, the signature of a loop—as the back-edge of a candidate loop, and takes the instructions from the target to the branch as that loop’s body. It scores each candidate by a compute weight that rewards the opcode families mining relies on (integer, bitwise, shift, predicate, and memory operations) along with loop size, back-edges, and nesting depth, and keeps the highest-scoring loop. When

Algorithm 1 Main computational loop extraction for one kernel.

Require: normalized instruction list I with branch targets

```

1:  $C \leftarrow \emptyset$  ▷ loop candidates
2: for all branches  $b \in I$  with target  $t$  before  $b$  do
3:    $body \leftarrow I[t : b]$  ▷ back-edge loop body
4:    $C \leftarrow C \cup \{(body, \text{SCORE}(body))\}$ 
5: end for
6: if  $C \neq \emptyset$  then
7:   return body of  $\arg \max_C \text{SCORE}$ 
8: else if  $|I| \leq \tau_{\text{small}}$  then
9:   return  $I$  ▷ short kernel: keep whole body
10: else
11:   return highest-scoring compute-heavy region of  $I$ 
12: end if
13:
14: function  $\text{SCORE}(body)$  ▷ compute weight, higher is more loop-like
15:   return  $|body| + \sum_{f \in \text{families}} w_f \cdot \text{count}_f(body) + w_{\text{edge}} + w_{\text{nest}} \cdot \text{depth}$ 

```

a kernel has no recoverable loop, it falls back to the whole kernel if the kernel is short, and otherwise to the highest-scoring compute-heavy straight-line region. Because the procedure uses only opcodes and branch targets, it is robust to register allocation and instruction scheduling.

GPU-Sentry normalizes operands to strip unstable details while preserving computation. It canonicalizes instance-specific operands such as registers, addresses, branch labels, and literals, and drops NOP instructions, which carry no computation. It preserves each instruction’s opcode and operand types, so the semantic families that matter to both mining and benign compute—integer and bitwise arithmetic, memory access, control flow, and floating-point or tensor math—remain distinguishable. Listing 1 shows the effect.

```

IADD3 R12, R13, c[0x0][0x20], RZ
-> IADD3 REG, REG, CONST, ZERO

@P0 BRA 0x1a0
-> @PRED BRA LABEL

```

Listing 1: SASS normalization preserves opcode and operand-type semantics while removing instance-specific detail.

GPU-Sentry also maps architecture-specific opcode variants to common semantic families, because GPU generations often express the same operation with different opcodes or qualifiers. Integer multiply-add, for instance, is XMD on Maxwell and Pascal but IMAD on Volta and later, and both map to IMAD; a warp shuffle is SHFL before Volta and SHFL.SYNC from Volta onward, and both map to SHFL. This keeps semantically equivalent integer, bitwise, memory, branch, and synchronization patterns comparable across compilers and GPU generations.

4.4 Dynamic Kernel-Launch Window Scheduler

A mining workload is usually a launch *sequence*, not a single isolated kernel: a miner launches its kernels repeatedly, and many mining algorithms spread their work across several distinct kernels.

Algorithm 2 Per-stream window scheduler, run on each kernel launch.

Require: per-stream state:

W : launch window, oldest to newest;
 c : total token cost currently in W ;
 M : multiset of live kernel ids in W ;
 S : composition signatures already emitted for the stream;

$B = 8190$: content-token budget

```

1: On kernel launch  $k$  on its stream:
2: append  $k$  to  $W$ ;  $c \leftarrow c + \text{TokenCost}(k)$ ; increment  $M[k.id]$ 
3: if  $c \leq B$  then
4:   return ▷ budget not full; keep collecting
5: end if
6:  $\sigma \leftarrow \langle \text{SORTEDSET}(M), \text{TOP3}(M) \rangle$  ▷ composition signature
7: if  $\sigma \notin S$  then
8:   emit  $\text{RENDER}(W)$  as one window ▷ composition not
   classified for the stream yet
9:    $S \leftarrow S \cup \{\sigma\}$ 
10: end if
11: while  $c > B$  do
12:   pop oldest launch  $k_0$  from  $W$ ;  $c \leftarrow c - \text{TokenCost}(k_0)$ ;
   decrement  $M[k_0.id]$ 
13: end while
14:
15: function  $\text{RENDER}(W)$  ▷ turn the window into one
   ModernBERT input
16:    $r \leftarrow \text{concat normalized kernels in } W, \text{ joined by}$ 
    $\text{KERNEL\_BOUNDARY}$ 
17:   return  $\text{KEEPLAST}(r, B)$  ▷ clip to  $B$  tokens, keep the newest

```

GPU-Sentry therefore analyzes streams of launches. It maintains a per-stream rolling window, grouping launches by process, device, and CUDA stream. Grouping by stream both preserves CUDA launch order within a workload and keeps unrelated work apart, so the repeated or split launches of one miner accumulate in the same window while every verdict stays attributable to one process. The detector consumes one fixed-size sequence at a time, so the scheduler’s job is to turn an unbounded launch stream into model-sized windows, each rendered to the content budget of 8190 tokens (reserving two of the model’s 8192 positions for the [CLS] and [SEP] tokens).

Algorithm 2 summarizes the per-stream window-scheduling logic; the paragraphs below explain its parts.

Filling and emitting a window. Each launch contributes a known token cost: its rendered normalized SASS fragment plus one KERNEL_BOUNDARY token. GPU-Sentry appends each launch to the stream’s window and accumulates its cost, collecting launches until the window fills the token budget. It then emits the full window for classification and evicts the oldest launches to make room for the next one. Waiting until the budget is full ensures every emitted window carries enough code to classify, rather than a tiny first-launch fragment.

Composition-signature deduplication. A steady stream fills and emits windows repeatedly with the same kernel mixture, so GPU-Sentry deduplicates emissions by composition signature: the sorted set of live kernel identities together with the three most

frequent kernels. A window is emitted only if its signature has not been emitted before for that stream, so a stream is classified when its kernel composition is new rather than on every full window.

Rendering and front clipping. The RENDER subroutine in algorithm 2 turns a window into one ModernBERT input: it concatenates the window’s normalized launch fragments joined by KERNEL_BOUNDARY tokens, and because a full window can slightly exceed the budget, clips from the front and keeps the newest fragments so the final input is exactly one model-sized sequence. The scheduler therefore owns the token budget and the detector classifies exactly one sequence per window. Listing 2 shows a rendered window.

```

; Normalized main loop for Kernel 1
IADD3 REG, REG, CONST, ZERO
LOP3 REG, REG, REG, CONST
SHF REG, REG, IMM
@PRED BRA LABEL
KERNEL_BOUNDARY
; Normalized main loop for Kernel 2
IMAD REG, REG, REG, ZERO
LDG REG, MEM
STG MEM, REG
KERNEL_BOUNDARY
...

```

Listing 2: A rendered window concatenates per-launch normalized SASS fragments, separated by KERNEL_BOUNDARY tokens, into one model-sized sequence.

4.5 ModernBERT Instruction-Sequence Detector

GPU-Sentry classifies each normalized instruction window with a learned model: an encoder-only ModernBERT model [22] used as a binary classifier over normalized instruction sequences. Because it is encoder-only, it attends over the whole token window at once, which makes it a fast detector and lets it learn both local instruction relationships within a kernel and recurrence patterns across launches. We use an encoder rather than a decoder-only LLM because detection is a classification task that needs no left-to-right text generation, which keeps inference cheaper for the same context length. Its input is normalized SASS joined by KERNEL_BOUNDARY tokens; scheduling metadata such as timestamps, launch dimensions, and process names are deliberately excluded so the model reasons about code, not context.

Because the scheduler already bounds every window to the content budget, each rendered window yields a single mining probability. A per-window threshold gives a diagnostic verdict, while the deployment decision is made by the workload policy below. ModernBERT is the primary and sole decision layer in this work (section 6).

4.6 Decision

The decision module separates detection output from deployment policy. Each classified window yields a mining score and the capture context needed for attribution, but a single window may not carry enough evidence for a confident decision, so GPU-Sentry aggregates per-window scores into a workload decision with a rolling policy

over the most recent windows of a stream (the last eight in our deployment). A workload is flagged when the rolling mean mining probability is at least 0.30 *and* the rolling maximum is at least 0.50 (section 6.5 describes how both thresholds are chosen). The mean term requires sustained mining-like evidence across the workload, while the max term keeps a benign workload with many steady, moderate scores from triggering on its average alone. When the policy fires, a deployment may notify an administrator through a notification channel or block the workload by sending a termination command to the client interception thread, on receipt of which the offending process self-terminates.

4.7 Archive

GPU-Sentry archives captured code objects, normalized artifacts, window metadata, model scores, and verdicts to disk. Unlike device telemetry, which can only show that the GPU was busy, instruction-level detection produces evidence that can be re-examined after an alert.

4.8 Implementation

The interception library is implemented in C++ together with a Go client bridge that exports events on a background thread; the two are statically linked into a single replacement libcuda.so.1. Events are serialized as Protobuf and sent to the collector over UDP. The collector server is implemented in Go, and offline analysis and ModernBERT training run in Python. The same capture path is used both to build datasets and to run deployment-mode experiments.

4.9 Extensibility: Optional Verification Layer

For deployments with stricter false-positive tolerance, GPU-Sentry can route uncertain windows to an optional second-stage verifier, for example a locally hosted LLM, that reasons jointly over the unnormalized main-loop code, kernel statistics, ModernBERT scores, and launch behavior, explicitly weighing benign crypto-like alternatives such as cryptographic libraries, compression, checksums, password-recovery benchmarks, and integer-heavy HPC. This layer is conceptual and unimplemented in this work: we report results from ModernBERT alone, so the verifier is never invoked in our experiments. It targets exactly the low-confidence borderline cases the detector currently mishandles, such as the single-window captures missed in our real-world evaluation, and we leave its implementation and evaluation to future work.

5 DATASET AND TRAINING

5.1 Synthetic-to-Real Transfer

Real GPU-miner datasets are too small to train a detector on directly and too difficult to collect at scale, so GPU-Sentry adopts a synthetic-to-real protocol: it learns from synthetic normalized SASS and is evaluated on real captures, which tests transferable instruction-level patterns rather than memorization. The tokenizer, the masked-language model, and the classifier are all trained and selected only on synthetic data; no real-world capture is used to train, validate, or select the learned detector, so the real-world dataset is evaluation-only. Only the workload policy’s two decision thresholds are set

Table 3: Synthetic normalized-SASS dataset used for training and synthetic validation/testing, with representative algorithms per class. Counts are total workloads (train / val / test).

Class	Representative algorithms	Total (tr/val/te)
Mining	Ethash, Etchash, ProgPoW, KawPoW, FiroPoW, CryptoNight, RandomX, Cuckaroo29, Cuckatoo31, Equihash, BeamHash III, Autolykos2, MTP, Octopus, Verthash, X16R, GhostRider, FishHash, KarlsenHash v2, kHeavyHash, SHA-256d, Blake3, Keccak	130 (91/20/19)
Benign compute	SGEMM/DGEMM, Conv2D, attention, softmax, LayerNorm, FFT, DCT, Sobel, PageRank, BFS, SSSP, bitonic sort, prefix scan, reduction, Huffman, LZ match, CRC32, xxHash	80 (56/12/12)
Benign crypto	SHA-256, SHA-512, Keccak, BLAKE2b, ChaCha20, AES, Merkle tree, Philox RNG	28 (20/4/4)
Benign memory	STREAM copy/scale/add/triad, pointer chase, gather/scatter, random-index read	14 (10/2/2)
Total		252 (177/38/37)

on the real-world distributions as deployment operating points (section 6.5); the learned model itself never sees real data.

5.2 Synthetic Workloads

The synthetic dataset contains 252 captured workloads, evenly split across O2 and O3 builds, with 130 mining-like and 122 benign workloads. The workloads were generated from algorithm lists and natural-language descriptions and implemented with the assistance of a large language model (GPT-5.5 [13]), and each algorithm is built both as a single-kernel (mono) variant and as a split variant that distributes work across helper kernels, so the detector sees both compact and split execution. Table 3 lists representative algorithms in each class together with the workload counts. Because the scheduler can emit more than one window per workload, the 252 workloads yield 361 windows under a grouped 70/15/15 split (253 train, 56 validation, 52 test), with all windows of a workload kept on the same side of the split.

The benign crypto/hash-like class is included deliberately as a hard negative. Cryptographic primitives and checksums can be dominated by the same integer and bitwise instructions that characterize proof-of-work hashing, so this class probes whether the detector has learned more than a naive “bitwise-heavy means mining” association.

Split policy. The synthetic train, validation, and test splits avoid leakage from shared lineage. Workloads that share a source algorithm, implementation, or binary are kept on the same side of a split rather than divided at random, and the O2 and O3 variants of a workload are kept together, so the model cannot trivially match a window to its optimization twin across splits.

5.3 Real-World Captures

The real-world dataset is collected through the same runtime capture layer used in deployment, and we build it into 58 evaluation-only workload groups: 48 mining-like and 10 benign. The mining captures come from production GPU miners and the benign captures from real GPU applications. Because some miners support many algorithms, we run each miner once per algorithm and treat

Table 4: Real-world evaluation dataset: captured programs and workload-group counts. Miners are captured per algorithm; all groups are evaluation-only.

Type	Programs (example algorithms)	Groups
Miners	T-Rex, GMiner, lolMiner, NBMiner, Rigel, XMRig (ethash, etchash, kawpow, progpow, octopus, autolykos2, kheavyhash, randomx)	48
Benign apps	PyTorch training, vLLM inference, HPL, Blender OptiX, cuBLAS GEMM, cuDNN convolution, GPU compression, CRC32 / SHA-256 / AES benchmarks	10
Total		58

Table 5: ModernBERT detector configuration.

Item	Value
Architecture	encoder-only, 8 layers, hidden 384, 6 heads
Max sequence length	8192 tokens
Tokenizer	word-level SASS, vocabulary 64
MLM pretraining	8 epochs, 15% masking, eval loss 0.083
Classifier training	10 epochs, binary

Table 6: Binary detector results on the synthetic validation and test splits (window-level). Workload-level aggregation is also 1.000 on both splits.

Split	Windows	Acc.	Macro F1
Validation	56	1.000	1.000
Test	52	1.000	1.000

each (miner, algorithm) run as a separate workload. Table 4 lists the captured programs and group counts.

5.4 ModernBERT Training

The detector is a compact encoder-only ModernBERT: 8 transformer layers, hidden size 384, 6 attention heads, and a maximum sequence length of 8192 tokens. Its inputs are normalized SASS, so a small word-level tokenizer with a 64-token vocabulary suffices. Training proceeds in three stages on the synthetic data. GPU-Sentry first trains the tokenizer on normalized SASS from the training split, then warms the model from scratch with masked-language modeling (8 epochs, 15% masking) to learn instruction semantics, and finally trains the binary classifier (10 epochs). Table 5 summarizes the configuration.

On the held-out synthetic windows the detector is perfect (table 6): window-level accuracy and macro F1 are 1.000 on both validation (56 windows) and test (52 windows), and aggregating windows to workloads is likewise perfect on validation, test, and the two combined. This clean synthetic fit, together with the hard-negative crypto class, is what makes the real-world captures the meaningful test of transfer (section 6).

6 EVALUATION

Our evaluation asks four questions: whether GPU-Sentry captures representative CUDA activity at low overhead, whether synthetic training transfers to real-world SASS windows, whether instruction-level detection resists launch-rate throttling, and whether per-stream windows survive mixed workloads, in each case compared against a telemetry baseline.

6.1 Experimental Setup

Experiments run on a server with two Intel Xeon Silver 4314 CPUs, 188 GiB RAM, and eight NVIDIA V100 GPUs, on Debian 12 with Linux 6.1.0, NVIDIA driver 580.65.06, and CUDA 12.8.

Baseline. We compare against a behavioral baseline drawn from Pott-style GPU-counter detection [14]: a Random Forest over whole-device NVML [12] point measurements (GPU utilization, memory utilization, power, and temperature). It predicts a mining probability per sample and raises a run-level alarm when the mean mining probability is at least 0.8, a threshold tuned on its own validation split with a maximum benign false-positive rate of 0.01. We restrict the comparison to detectors that, like GPU-Sentry, reason about GPU execution; network, host-signature, and side-channel detectors target a different signal and are out of scope here.

6.2 Capture Effectiveness

GPU-Sentry must first show that transparent capture observes real CUDA activity across benign and mining workloads. Table 7 reports the three most frequently launched named kernels in representative vLLM, HPL, and T-Rex captures, each chosen to stress a different capture path: vLLM runs on the PyTorch framework, HPL (the NVIDIA HPC Benchmarks 21.4 image from NGC) runs inside a Docker container, and T-Rex uses binary obfuscation and anti-debugging techniques. GPU-Sentry captures AI serving kernels, dense linear algebra kernels, and mining search kernels through the same Driver-API path. The HPL case shows that capture reaches containerized workloads, and the T-Rex case is the important one for evasion: even though T-Rex obfuscates its host process, its GPU computation must still traverse the Driver API, so GPU-Sentry captures its `cuda_ethash_search` kernels and their SASS.

6.3 Capture Overhead

GPU-Sentry adds a small latency to CUDA Driver-API calls (table 8, medians over 100 runs of a vector-add program). The jump-only `cuDeviceGetCount` trampoline adds about 0.13 μ s, module load adds about 68.7 μ s because GPU-Sentry records the loaded code object, and kernel launch adds about 9.8 μ s. Asynchronous buffering is what keeps the launch path cheap. We compare against our previous capture system, cuPCAP [4], which exports each event synchronously on the intercept path and, lacking trampolines, wraps every forwarded call in an extra stack frame: it adds about 16.0 μ s per launch, so GPU-Sentry’s in-process ring buffer cuts kernel-launch overhead by roughly 40% (9.8 versus 16.0 μ s). The same asynchrony keeps module-load and trampoline latency far below the synchronous design across all three calls.

At the application level, the overhead is not measurable for the representative workloads (table 9). HPL reports 3608 GFLOP/s without capture and 3721 GFLOP/s with capture, within run-to-run

Table 7: Top launched named kernels from representative captures, each exercising a different capture challenge. Compute ratio is the fraction of parsed SASS instructions in compute opcode families.

Workload	Capture challenge	Kernel	Launches/s	Instr.	Compute ratio
vLLM	PyTorch framework	reshape_and_cache_kernel_flash	132.758	50	0.680
		triton_red_fused_...	127.371	158	0.753
		triton_poi_fused_...	127.320	143	0.881
HPL	Containerized	volta_dgemm_128x64_nt	42.471	1054	0.793
		trsm_right_kernel<... ,true>	41.683	198	0.576
		trsm_right_kernel<... ,false>	41.683	184	0.516
T-Rex	Obfuscation & anti-debug	cuda_ethash_search	57.796	1512	0.854
		cuda_ethash_search_1	29.190	1555	0.849
		cuda_ethash_search_2	29.044	2088	0.800

Table 8: Median CUDA Driver-API latency (μ s): native, our prior synchronous capture design cuPCAP, and GPU-Sentry’s asynchronous capture; parentheses give the latency added over native. Asynchronous buffering lowers the added latency, most importantly for kernel launch.

API call	Native	cuPCAP (sync)	GPU-Sentry (async)
cuDeviceGetCount	2.575	6.545 (+3.970)	2.706 (+0.131)
cuModuleLoad	133.819	402.839 (+269.020)	202.484 (+68.665)
cuLaunchKernel	25.302	41.292 (+15.990)	35.081 (+9.780)

Table 9: Application throughput with and without capture. Overhead is the relative throughput loss (native – capture)/native; negative values mean capture measured slightly faster, within run-to-run variation.

Workload	Metric	Native	Capture	Overhead (%)
HPL	GFLOP/s	3608	3721	–3.1
T-Rex	MH/s	90.75	90.75	0.0
vLLM	tokens/s	1827.68	1827.91	–0.01

variation; T-Rex reports an identical 90.75 MH/s; and vLLM reports 1827.68 versus 1827.91 tokens/s. Because module loads are rare relative to kernel launches and launches are buffered asynchronously, capture stays off the throughput-critical path.

6.4 Synthetic-to-Real Detection

Because the learned detector is trained only on synthetic data, we evaluate it on all 58 real-world workload groups, applying the same rolling workload policy used online (section 4.6). This yields the confusion matrix in table 10 and the metrics in table 11: GPU-Sentry reaches 0.966 accuracy and 0.944 macro F1, flagging 46 of 48 mining workloads (0.958 recall) with no benign false positives (1.000 mining precision, 1.000 benign recall). The two missed workloads are the Pyrin-family miners lolminer_pyrin and lolminer_pyrinv2, whose matrix-hash code the detector scores as benign (mining probability 0.027 and 0.100); we analyze this representation limit, which no threshold can fix, in section 6.5.

Table 10: Real-world workload-level detection for GPU-Sentry over all 58 workload groups under the rolling workload policy. Rows are ground truth, columns are predictions.

True label	Pred. benign	Pred. mining
Benign	10	0
Mining	2	46

Table 11: Real-world workload-level detection metrics over all 58 workload groups.

Class	Workloads	Precision	Recall	F1
Benign	10	0.833	1.000	0.909
Mining	48	1.000	0.958	0.979
Accuracy			0.966 (56/58)	
Macro F1				0.944

The benign side is clean because the detector scores benign workloads low and the workload policy requires sustained, high-confidence evidence: every benign application, including compute-bound ones such as cuBLAS GEMM, cuDNN convolution, HPL, and Blender OptiX, stays below the policy thresholds (section 6.5).

Comparison with the behavioral baseline on basic runs.

The behavioral baseline is just as accurate on these ordinary isolated runs, which is why telemetry detectors are attractive in practice. We score the same real-world workloads with the synthetic-trained Random Forest under the same synthetic-to-real transfer, replaying each workload under whole-device telemetry capture without retraining. On this set of 48 mining and 10 benign workloads the baseline reaches 0.979 mining recall with no benign false positives (tables 12 and 13), level with GPU-Sentry’s 0.958 from table 11. Both detectors are therefore strong on ordinary runs; the contrast emerges only under launch-rate throttling and mixed workloads (figure 6 and table 14), where the baseline collapses while GPU-Sentry stays at 1.000. The two detectors also fail on different axes: the baseline’s single miss is the RandomX miner xmr_ig, whereas GPU-Sentry’s are the Pyrin-family matrix-hash miners.

Table 12: Basic-run detection on the same real-world isolated workloads (48 mining, 10 benign). Both detectors are accurate on ordinary runs; the gap appears only under throttling and mixed workloads. The behavioral baseline replays each workload under telemetry and scores it with the synthetic-trained Random Forest; the GPU-Sentry row repeats table 11 for side-by-side comparison.

Detector	Mining recall	Benign recall	Accuracy
Behavioral baseline	0.979 (47/48)	1.000 (10/10)	0.983 (57/58)
GPU-Sentry	0.958 (46/48)	1.000 (10/10)	0.966 (56/58)

Table 13: Behavioral baseline confusion matrix on the real-world isolated workloads (10 benign, 48 mining). Rows are ground truth, columns are predictions.

True label	Pred. benign	Pred. mining
Benign	10	0
Mining	1	47

6.5 Threshold Selection

GPU-Sentry’s only tuned decision point is the rolling workload policy (section 4.6): the learned detector is trained and selected on synthetic data, while the policy’s two scalar thresholds are chosen from the real-world workload distributions in figure 5.

The workload policy flags a stream only when its rolling mean mining probability is at least 0.30 *and* its rolling maximum is at least 0.50. Figure 5 plots each real-world workload’s mean and maximum mining probability. No benign workload enters the flagged region. A maximum-only rule would fail: three benign workloads (vLLM, AES, and compression) each contain a single window the model scores above 0.93, which it would flag as mining. The mean term is what rejects these false positives: averaged over each workload’s many other low-scoring windows, their rolling mean stays below 0.23, well under the 0.30 threshold. A mean-only rule, conversely, would miss borderline miners whose evidence is concentrated in a few windows, so the maximum term keeps them flagged. The only two mining workloads that fall outside the flagged region, near the origin, are lolminer_pyrin and lolminer_pyrinv2; their low scores are a fundamental representation limit rather than a threshold artifact, discussed below.

The Pyrin matrix-hash case. The two missed miners, lolminer_pyrin and lolminer_pyrinv2, are not threshold failures: their mean and maximum mining probabilities are 0.027 and 0.100, far below any usable threshold. Pyrin’s proof-of-work is a matrix-multiplication hash deliberately designed to be ASIC-resistant by mapping the hash onto the same dense integer matrix-multiply primitives that GPUs and AI accelerators are built for. The resulting SASS is dominated by integer dot-product (IDP) instructions and is, at the instruction level, essentially indistinguishable from a benign integer-GEMM or quantized-inference kernel. This is an inherent limit of code-level detection for matmul-style mining algorithms, not a tuning choice, and it motivates the optional verification layer and runtime-context signals (section 4.9).

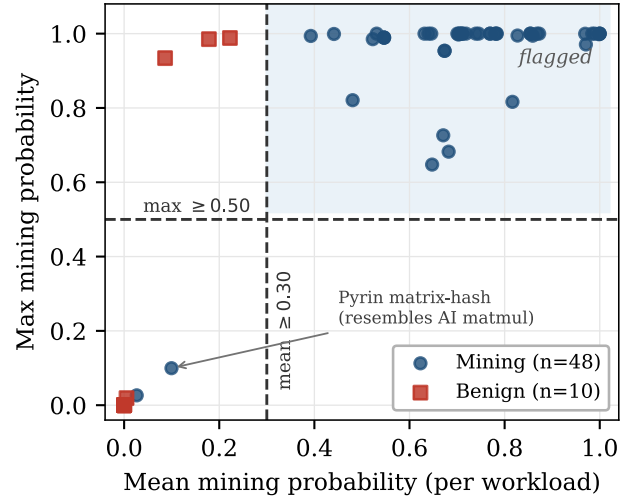


Figure 5: Per-workload mean versus maximum mining probability on the real-world set. The flagged region (shaded) requires mean ≥ 0.30 and max ≥ 0.50 . Benign workloads with one spiky window sit top-left (high max, low mean) and are excluded by the mean term; no benign workload is flagged. The two Pyrin-family miners near the origin are scored benign by the detector because their matrix-hash SASS resembles AI matmul.

6.6 Launch-Rate Throttling

Throttling the kernel-launch loop affects telemetry far more than instruction-level detection (figure 6). GPU-Sentry evaluates 288 windows from throttled captures across all synthetic mining algorithms, at 10%, 50%, 75%, and 100% launch rates, and classifies all 288 as mining (recall 1.000 at every rate). Throttling changes launch timing, not the normalized SASS body or the token cost each launch contributes, so a throttled stream still fills the budget and the emitted window still contains the mining kernels; it just fills more slowly.

The behavioral baseline detects every run at the 5% and 100% settings but misses every run at 25%, 50%, and 75% and one of eight at 10%. Mid-rate throttling moves utilization and power into ordinary-looking bands where the device counters no longer look mining-like. The 5% and 100% recoveries have opposite causes. At 100% the device shows its native full-rate mining footprint, which the baseline learned as mining. At the 5–10% extreme, by contrast, heavy throttling collapses the footprint to a near-idle range (≈ 5 – 10% utilization, ≈ 40 W) that lies outside the busy benign workloads the Random Forest was trained on; the classifier therefore flags these runs as “not a heavy benign job” with high confidence rather than recognizing mining as such. This is brittle: the same boundary would raise false alarms on any genuinely idle or lightly loaded GPU, so it is not the robustness it appears to be. The mid rates instead place the device squarely in the ordinary busy-workload band that overlaps benign jobs, where confidence collapses and the alarm never fires. This is the sharpest contrast between the two approaches: GPU-Sentry is invariant to launch-rate throttling because it reads the kernel body.

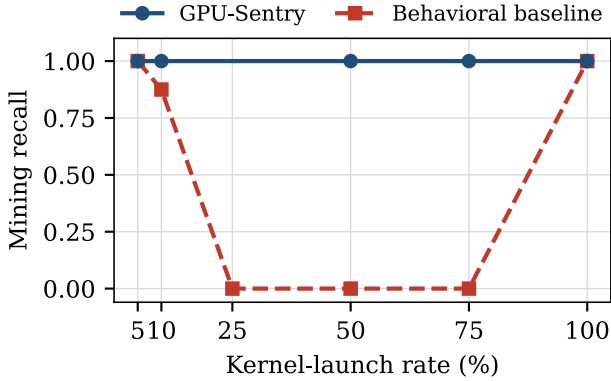


Figure 6: Mining recall versus kernel-launch throttling. GPU-Sentry stays at 1.000 across the evaluated rates (5–100%), while the GPU-counter behavioral baseline collapses to 0 at mid rates (25–75%) and recovers only at the 5% and 100% extremes. Aggregated, GPU-Sentry detects 288/288 windows (recall 1.000) and the baseline 23/48 runs (recall 0.479).

Table 14: Mixed-workload runs. Each run co-locates a benign GPU workload with a throttled miner on one device. Whole-device counters miss most miners and cannot attribute alarms to a process; GPU-Sentry’s per-stream windows isolate and detect every mining stream. The cuBLAS GEMM run was repeated, for seven runs total.

Benign co-tenant	Miner	Behavioral baseline	GPU-Sentry
cuBLAS GEMM	RandomX	×	✓
cuDNN convolution	KawPoW	×	✓
HPL (Linpack)	SHA-256d	×	✓
PyTorch training	Ethash	✓	✓
AES	Ethash	×	✓
vLLM inference	RandomX	✓	✓
Mining recall		0.286 (2/7)	1.000 (7/7)

6.7 Mixed Workloads

When a benign workload and a throttled miner share one GPU, whole-device telemetry loses the miner, but instruction-level detection does not. We co-locate each of seven mining algorithms with a representative benign GPU workload on one device (table 14). The behavioral baseline detects only two of the seven, because the benign workload dominates the shared device counters. It also has no PID or stream attribution, so even when it does alarm it cannot point to the offending process. GPU-Sentry detects all seven mining streams. Its windows are per process and per stream by construction, so the benign stream and the mining stream remain separate detection units: the mining stream emits the same normalized SASS window it would emit on its own, regardless of the co-running benign job, and the detector classifies it independently and attributes it to the offending PID and stream. Sharing the device therefore changes neither the kernel body nor GPU-Sentry’s decision, which is why mixing leaves its recall unchanged while it collapses the telemetry baseline.

7 LIMITATIONS

Userspace enforcement boundary. GPU-Sentry’s main deployment limitation is that it runs in a userspace library, so a same-privilege user can bypass it—by shipping their own `libcuda.so.1`, calling the driver directly, or tampering with the monitored library. Moving the check into the kernel does not work either: once a program has a GPU channel, it submits each launch by writing directly to a command buffer that the kernel never sees. The kernel does still hand out those channels in the first place, so NVIDIA’s open-source GPU kernel modules make enforcement possible: an administrator could allow a GPU channel to be created only through the monitored path, leaving a user no way to obtain an unmonitored one. We leave this to future work.

Code obfuscation. Obfuscation can also weaken instruction-level detection by changing the instruction stream enough to hide mining-like structure. On a GPU this is unusually costly: a mining kernel is the workload’s core computation, hand-tuned for throughput through register allocation, occupancy, memory-access patterns, and instruction scheduling, so inserting instructions or perturbing its control flow degrades exactly the performance the miner depends on. Because mining revenue scales with hash throughput, obfuscation trades away profit, and no GPU-code obfuscation tooling exists for this purpose today. GPU-Sentry does not assume obfuscation is impossible; it treats obfuscated or low-confidence cases as a reason for additional verification and audit rather than as solved.

Benign-looking mining algorithms. The sharpest limit is a proof-of-work that is, by construction, the same computation as a benign GPU kernel. Matrix-multiplication proof-of-work such as Pyrin is designed to be ASIC-resistant by mapping the hash onto the same dense integer matrix-multiply primitives that GPUs and AI accelerators are optimized for, so its SASS is essentially indistinguishable from benign integer GEMM or quantized inference (section 6.5). Code-level detection alone cannot separate such workloads from legitimate AI compute; distinguishing them requires runtime context or a second-stage verifier (section 4.9) rather than the instruction window alone.

8 RELATED WORK

GPU cryptojacking detection. Prior GPU cryptojacking detectors reason about behavior, telemetry, or counters: decision trees over host and GPU metrics [18], classifiers over GPU device telemetry [14], and physical side-channel monitors [1, 23]. These methods are practical because they need no code inspection, but they infer intent from effects such as utilization, power, repeated launches, or counter patterns, which we show degrade under throttling and mixed workloads. GPU-Sentry instead captures and classifies the submitted GPU instruction stream; to our knowledge, it is the first GPU-cryptomining detector to use the submitted GPU instructions (SASS) as its detection signal rather than these indirect effects.

GPU binary instrumentation and capture. GPU binary instrumentation frameworks such as NVBit [20] and SASSI [16] can observe execution at instruction granularity, and observability tools such as DCGM, Nsight Systems, and CUPTI expose rich runtime

signals [7–9]. These tools target profiling, debugging, and cluster monitoring rather than deciding whether a SASS window implements mining, and per-instruction instrumentation is heavier than continuous monitoring needs. GPU-Sentry captures code-load and launch events at the Driver-API boundary, trading instruction-by-instruction tracing for transparent, low-overhead deployment across native and containerized workloads.

Static binary analysis. Static malware and cryptojacking analysis [19] show that program semantics are useful security evidence, but host-binary analysis operates on CPU binaries and scripts that may be packed, renamed, or framework-generated, and it does not reveal the final GPU SASS produced by CUDA libraries or PTX JIT [3, 15]. GPU-Sentry applies sequence learning to normalized GPU assembly [5, 21], where CUDA launch order and stream context are part of the representation.

Network and host telemetry. Network and host-telemetry detectors inspect mining-pool traffic [2] or host-level behavior [17]. They can be evaded by proxying, tunneling, traffic shaping, process renaming, and binary obfuscation, none of which change the GPU kernels that must execute. GPU-Sentry is complementary to these signals and remains effective when they are obscured.

9 CONCLUSION

We presented GPU-Sentry, an instruction-level detector for cryptomining-like GPU computation. Rather than inferring mining from network, host, telemetry, or side-channel effects, GPU-Sentry inspects the normalized SASS submitted to the GPU, preserves stream-level launch order, and classifies instruction windows. This code-level evidence stays visible when process and kernel names change, when host binaries are obfuscated, when launch rates are throttled, and when benign work shares the device.

The evaluation supports three findings. First, the transparent CUDA Driver-API capture layer observes representative benign and mining workloads and adds small API latency with no measurable application slowdown for HPL, T-Rex, and vLLM. Second, a ModernBERT detector trained only on synthetic normalized SASS, where it separates mining from benign perfectly, transfers to 58 real-world workloads, flagging mining at 0.958 recall with no benign false positives; the only two misses are Pyrin-family matrix-hash miners whose code is, by design, indistinguishable from benign AI matrix multiplication. Third, instruction-level detection stays robust under kernel-launch throttling and mixed workloads: GPU-Sentry detects every throttled and co-running mining stream and attributes it to a process and stream, whereas a whole-device telemetry baseline misses most of them and cannot attribute the rest. The main next step is hardening the deployment against userspace driver bypass.

REFERENCES

- [1] Hangcheng Cao, Guowen Xu, Ziyang He, Shaoqing Shi, Shengmin Xu, Cong Wu, and Jianting Ning. 2025. Unveiling the Superiority of Unsupervised Learning on GPU Cryptojacking Detection: Practice on Magnetic Side Channel-Based Mechanism. *IEEE Transactions on Information Forensics and Security* 20 (2025), 4874–4889. <https://doi.org/10.1109/TIFS.2025.3563069>
- [2] Maurantonio Caprolu, Simone Raponi, Gabriele Oligeri, and Roberto Di Pietro. 2021. Cryptomining Makes Noise: Detecting Cryptojacking via Machine Learning. *Computer Communications* 171 (2021), 126–139. <https://doi.org/10.1016/j.comcom.2021.02.016>
- [3] Ari B. Hayes, Fei Hua, Jin Huang, Yanhao Chen, and Eddy Z. Zhang. 2019. Decoding CUDA Binary. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO '19)*. IEEE, 229–241. <https://doi.org/10.1109/CGO.2019.8661186>
- [4] Pin-Yi Kuo and Jerry Chou. 2025. cuPCAP: Non-intrusive GPU Activity Capture for Security Auditing on CUDA Platforms. In *Proceedings of the 2025 Workshop on Analytics, Telemetry, and Cybersecurity for HPC (WATCH '25)*. Association for Computing Machinery, 8. <https://doi.org/10.1145/3733826.3762676>
- [5] Xuezixiang Li, Yu Qu, and Heng Yin. 2021. PalmTree: Learning an Assembly Language Model for Instruction Embedding. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. Association for Computing Machinery, 3236–3251. <https://doi.org/10.1145/3460120.3484587>
- [6] NVIDIA Corporation. 2022. NVIDIA CUDA Samples - cuHook (v11.8). https://github.com/NVIDIA/cuda-samples/tree/v11.8/Samples/2_Concepts_and_Techniques/cuHook. Accessed: 2026-06.
- [7] NVIDIA Corporation. 2025. CUDA Profiling Tools Interface (CUPTI). <https://developer.nvidia.com/cupti>. Accessed: 2026-06.
- [8] NVIDIA Corporation. 2025. NVIDIA Data Center GPU Manager (DCGM). <https://developer.nvidia.com/dcgmm>.
- [9] NVIDIA Corporation. 2025. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>.
- [10] NVIDIA Corporation. 2026. *CUDA Binary Utilities*. NVIDIA Corporation. <https://docs.nvidia.com/cuda/cuda-binary-utilities/> Includes nvdisasm and cuobjdump documentation; accessed: 2026-06-20.
- [11] NVIDIA Corporation. 2026. *CUDA Driver API*. NVIDIA Corporation. <https://docs.nvidia.com/cuda/cuda-driver-api/> Accessed: 2026-06-20.
- [12] NVIDIA Corporation. 2026. *NVIDIA Management Library*. NVIDIA Corporation. <https://developer.nvidia.com/management-library-nvml> Accessed: 2026-06-20.
- [13] OpenAI. 2026. GPT-5.5. <https://openai.com/>. Large language model used to assist synthetic workload generation. Accessed: 2026-06.
- [14] Claudius Pott, Berk Gulmezoglu, and Thomas Eisenbarth. 2023. Overcoming the Pitfalls of HPC-based Cryptojacking Detection in Presence of GPUs. In *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy (CODASPY '23)*. Association for Computing Machinery, 177–188. <https://doi.org/10.1145/3577923.3583655>
- [15] Jonas Roels, Adriaan Jacobs, and Stijn Volckaert. 2025. CUDA, Woulda, Shoulda: Returning Exploits in a SASS-y World. In *Proceedings of the 18th European Workshop on Systems Security (EuroSec '25)*. Association for Computing Machinery, 40–48. <https://doi.org/10.1145/3722041.3723099>
- [16] Mark Stephenson, Siva Kumar Sastry Hari, Yunusup Lee, Eiman Ebrahimi, Daniel R. Johnson, David Nellans, Mike O'Connor, and Stephen W. Keckler. 2015. Flexible Software Profiling of GPU Architectures. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15)*. Association for Computing Machinery, 185–197. <https://doi.org/10.1145/2749469.2750375>
- [17] Rashid Tahir, Muhammad Huzaifa, Anupam Das, Mohammad Ahmad, Carl A. Gunter, Fareed Zaffar, Matthew Caesar, and Nikita Borisov. 2017. Mining on Someone Else's Dime: Mitigating Covert Mining Operations in Clouds and Enterprises. In *Research in Attacks, Intrusions, and Defenses (RAID 2017) (Lecture Notes in Computer Science)*, Vol. 10453. Springer, 287–310. https://doi.org/10.1007/978-3-319-66332-6_13
- [18] Dmitry D. Tanana. 2024. Behavior-Based Detection of GPU Cryptojacking. In *2024 IEEE 3rd International Conference on Problems of Informatics, Electronics and Radio Engineering (PIERE)*. IEEE, 416–419. <https://ieeexplore.ieee.org/document/10804931>
- [19] Ege Tekiner, Abbas Acar, A. Selcuk Uluagac, Engin Kirda, and Ali Aydin Selcuk. 2021. SoK: Cryptojacking Malware. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 120–139. <https://doi.org/10.1109/EuroSP51992.2021.00019>
- [20] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. 2019. NVBit: A Dynamic Binary Instrumentation Framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '52)*. Association for Computing Machinery, 372–383. <https://doi.org/10.1145/3352460.3358307>
- [21] Hao Wang, Wenjie Qu, Gilad Katz, Wenyu Zhu, Zeyu Gao, Han Qiu, Jianwei Zhuge, and Chao Zhang. 2022. jTrans: Jump-Aware Transformer for Binary Code Similarity Detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2022)*. Association for Computing Machinery, 1–13. <https://doi.org/10.1145/3533767.3534367>
- [22] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. 2024. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. *arXiv preprint arXiv:2412.13663* (2024). <https://doi.org/10.48550/arXiv.2412.13663>
- [23] Rui Xiao, Tianyu Li, Soundarya Ramesh, Jun Han, and Jinsong Han. 2023. Mag-Tracer: Detecting GPU Cryptojacking Attacks via Magnetic Leakage Signals. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (MobiCom '23)*. Association for Computing Machinery, 1027–1041. <https://doi.org/10.1145/3570361.3613283>